

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design

Weightage: 20%

QUESTIONS:

Total Marks: 25

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.
2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.
3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.

4.An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.

5.A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

I _____CH.Bhavya Sri/192411249_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

CH.Bhavya Sri

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted

Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect

1. REST vs SOAP – Selection and Design

Introduction

For a mobile and web application, both **REST** and **SOAP** can provide web services. However, **REST is the more suitable choice** when the application requires high performance, scalability, simple integration, and efficient communication.

REST vs SOAP Comparison

Feature	REST	SOAP
Architecture	Resource-based	Protocol-based
Data format	Mainly JSON; also XML	XML
Performance	Generally faster and lightweight	More overhead
Scalability	Highly scalable	Scalable but comparatively heavier
Mobile applications	Very suitable	Less convenient
Security	HTTPS, OAuth, JWT, etc.	HTTPS + WS-Security
Communication	HTTP methods	SOAP messages
Complexity	Simple	More complex
Caching	HTTP caching can be used	Less straightforward
Best suited for	Mobile apps, web APIs, cloud services	Enterprise systems requiring formal contracts and WS-* features

Selected Approach: REST

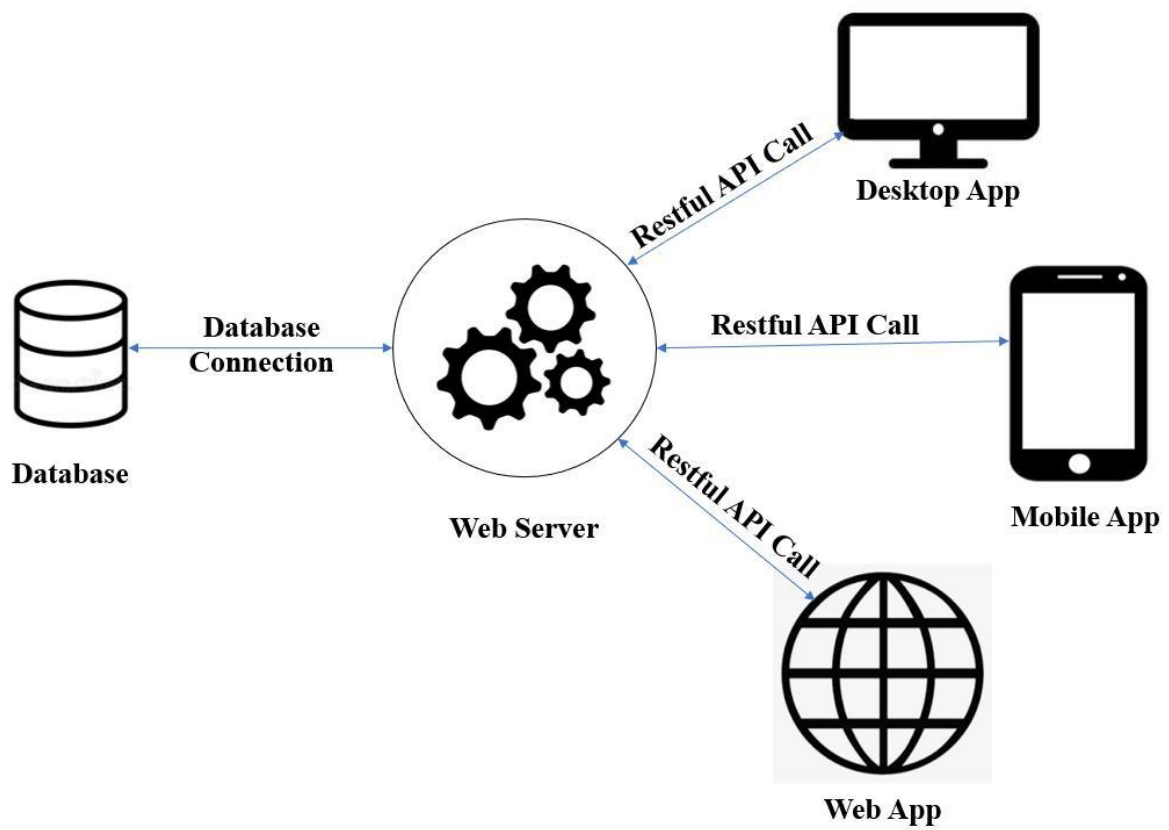
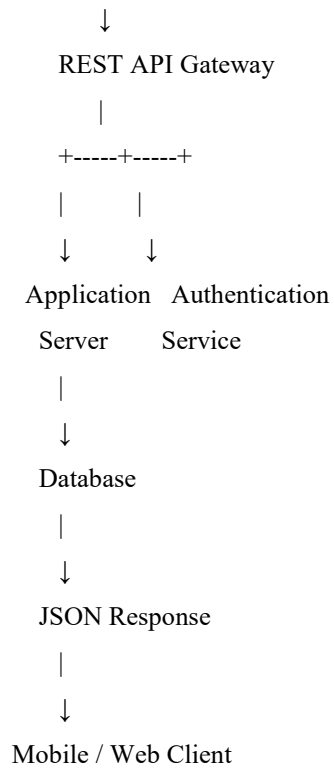
I would select **REST API** for the mobile and web application.

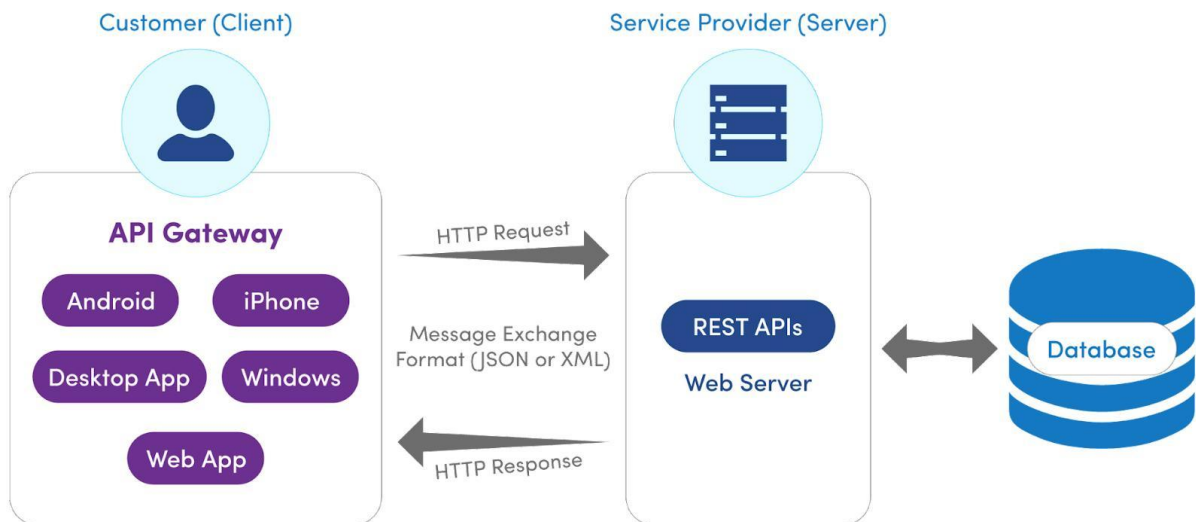
Proposed Architecture

Mobile App / Web App

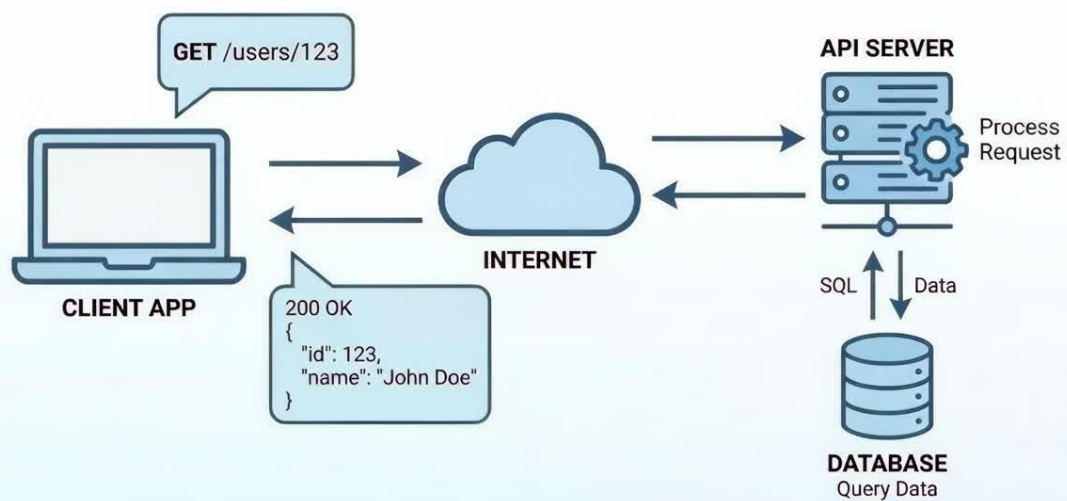
|

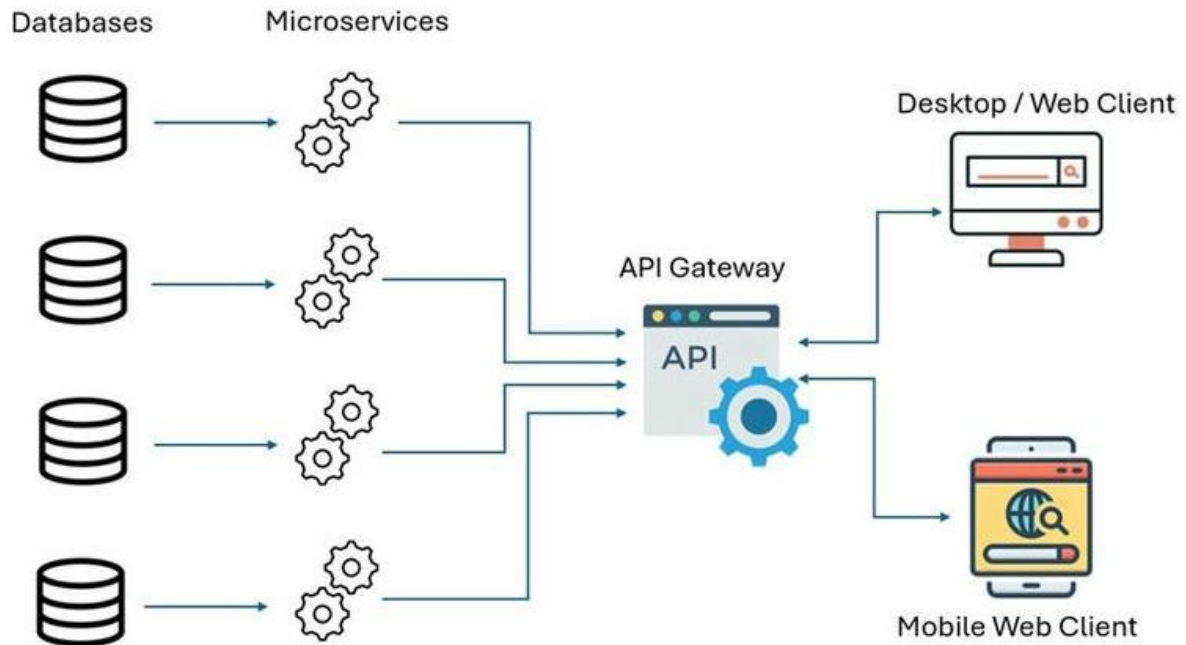
| HTTPS Request





REST API REQUEST FLOW DIAGRAM





Example REST Request

GET /api/products/101

Example Response

```
{  
  "id": 101,  
  "name": "Laptop",  
  "price": 55000  
}
```

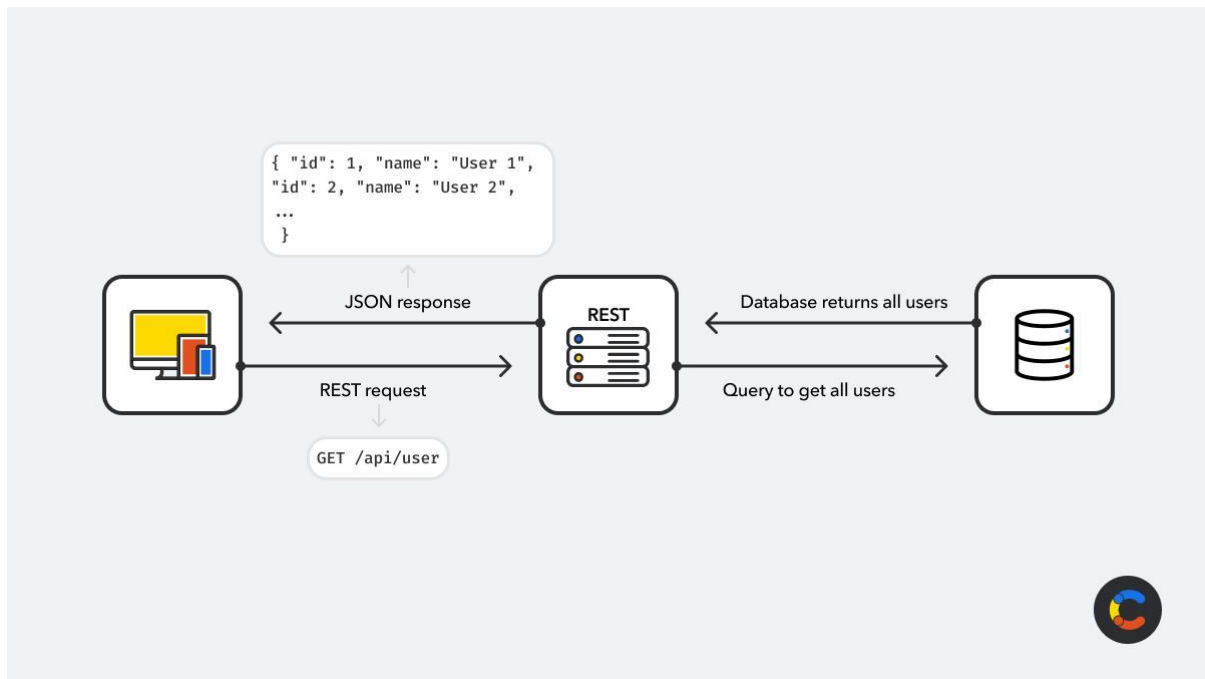
Justification

1. Performance:

REST commonly uses JSON, which is lightweight and requires less processing and bandwidth than SOAP's XML envelopes.

2. Scalability:

REST services can be designed as stateless services, allowing requests to be distributed across multiple servers using load balancing.



3. Security:

HTTPS provides encrypted communication, while authentication and authorization can be implemented using mechanisms such as OAuth 2.0 or tokens such as JWT.

4. Mobile compatibility:

REST APIs are easy to consume from Android, iOS and web applications.

5. Simplicity:

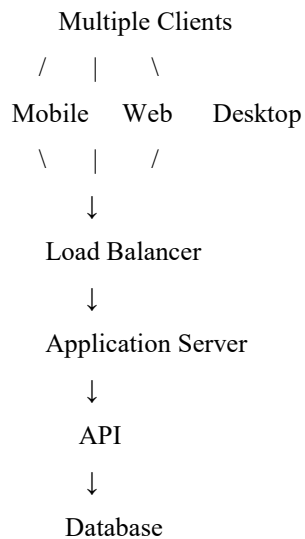
Standard HTTP methods such as GET, POST, PUT and DELETE make REST APIs relatively simple to develop and consume.

Conclusion

For a **mobile and web application**, REST is the better general-purpose choice because it provides lightweight communication, good scalability, strong performance and easy client integration. SOAP would be preferable when specialized enterprise features such as WS-Security or formal SOAP-based contracts are essential.

2. Scalable Service Architecture

Basic Architecture



Working

Clients send requests to the application.

The load balancer distributes requests among available servers.

The API receives and validates requests.

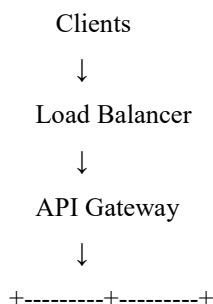
Application servers process business logic.

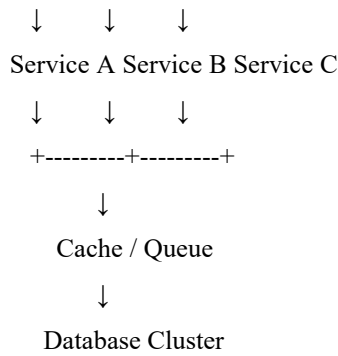
The database stores and retrieves required information.

The response is returned to the client.

Alternative Scalable Design

A better design for very high traffic is a **distributed architecture**:





Advantages

Multiple application servers prevent a single-server bottleneck.

Load balancing distributes traffic.

Caching reduces repeated database queries.

Message queues can handle background tasks asynchronously.

Database replication can improve availability and read performance.

Services can be scaled independently.

Conclusion: For large numbers of simultaneous users, a load-balanced, stateless API architecture with caching and database scaling provides better scalability than a single application server.

3. Secure Communication for Banking Application

Protocol Comparison

Protocol	Security	Reliability	Performance	Suitability
HTTP	Low without encryption	Good	High	Not suitable for banking
HTTPS	High with TLS	High	Good	Highly suitable
FTP	Not designed for web	Good	Good	Not suitable

Protocol	Security	Reliability	Performance	Suitability
	transactions			
SMTP	Email communication	Good	Moderate	Not suitable
WebSocket + TLS	High	High	Very good	Useful for real-time communication

Selected Protocol: HTTPS

HTTPS (HTTP Secure) should be selected for communication between the banking client and server.

Banking App

|

| HTTPS

| TLS Encryption

↓

Banking Web Server

|

↓

Authentication

|

↓

Banking Database

Justification

Security:

HTTPS uses **TLS** to encrypt data during transmission and helps protect sensitive information from interception and tampering.

Reliability:

HTTPS provides reliable communication over TCP and supports secure request-response communication.

Performance:

Modern TLS implementations have relatively low overhead, making HTTPS practical for banking applications.

Additional Security:

The application should also use strong authentication, authorization, secure session management, input validation and appropriate transaction controls.

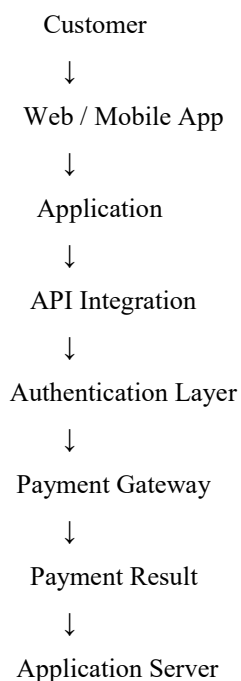
Conclusion

HTTPS with TLS is the most suitable standard protocol for banking client-server communication because it provides confidentiality, integrity and server authentication while maintaining good performance.

4. API Integration with Third-Party Services

Proposed API Integration Strategy

Suppose an e-commerce application needs to integrate with a **payment gateway**.



↓
Customer

Request Handling

User selects a payment method.

Application creates a payment request.

Required transaction details are validated.

The application authenticates with the third-party API.

The request is sent over HTTPS.

The third-party service processes the request.

The application receives the response.

The response is validated.

Transaction status is stored in the database.

The result is displayed to the user.

Authentication

Depending on the third-party service, authentication can use:

API keys

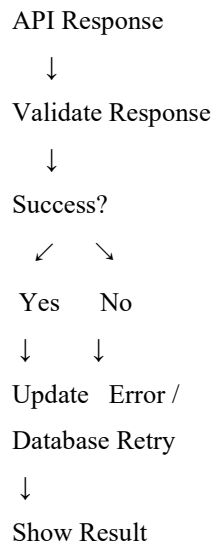
OAuth 2.0

Access tokens

Signed requests

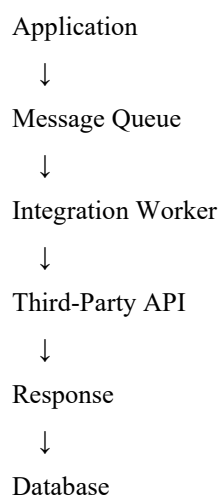
Sensitive credentials should be stored securely on the server and not exposed in client-side code.

Response Processing



Alternative Approach

Use an **asynchronous message queue** for operations that do not require an immediate response.



This prevents a slow third-party service from blocking the main application and allows controlled retries.

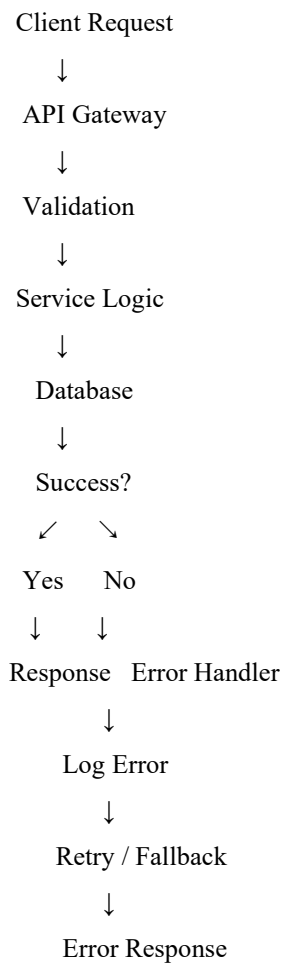
Conclusion

A secure API integration layer with authentication, validation, logging and retry mechanisms provides reliable third-party integration. For slow or failure-prone services, asynchronous processing can further improve reliability.

5. Error Handling and Fault Tolerance

Proposed Mechanism

A web service should detect errors, return meaningful responses, log failures and recover whenever possible.



Error Handling Techniques

1. Input Validation

Validate requests before processing them to prevent invalid data from reaching backend services.

2. Proper Error Codes

Return appropriate HTTP status codes such as:

400 – Bad Request

401 – Unauthorized

403 – Forbidden

404 – Not Found

500 – Internal Server Error

503 – Service Unavailable

3. Exception Handling

Catch unexpected application errors and return a controlled response rather than exposing internal details.

4. Logging and Monitoring

Record errors, timestamps, request IDs and service information so failures can be investigated.

5. Retry Mechanism

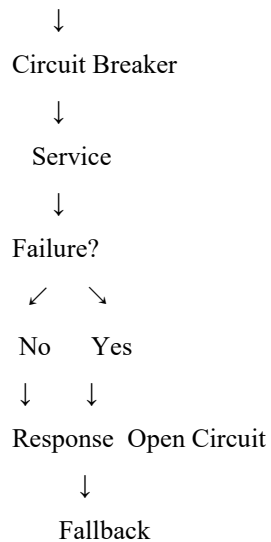
Temporary failures can be retried using controlled retry policies, preferably with exponential backoff.

6. Timeout

Set time limits for external services so that one slow service does not block the entire application.

7. Circuit Breaker

Service Request



A circuit breaker temporarily stops sending requests to a failing service and allows the system to recover.

Alternative Strategies

Caching: Serve frequently requested data from cache.

Fallback services: Provide an alternative response when a dependent service fails.

Message queues: Process non-critical operations asynchronously.

Load balancing: Distribute requests across multiple servers.

Replication: Maintain multiple instances of important services or databases.

Conclusion

A combination of **validation, meaningful error responses, logging, timeouts, controlled retries, circuit breakers, caching and fallback mechanisms** provides fault tolerance and improves the user experience even when individual components fail.